

Master's thesis

Evaluation of an Automated Validation Framework for
AI-Generated AWS CloudFormation Templates

Doshisha University
Graduate School of Science and Engineering
Information and Computer Science
Year 2024, Student 1316243106
Yerrapothu Venkata Kesava Siva Sai

Supervisor
Graduate School of Science and Engineering
Information and Computer Science
Network Information Systems Laboratory
Professor KOITA Takahiro

2026 July 20th

Abstract

Cloud infrastructure management has become a critical skill in modern software engineering. As organisations increasingly adopt cloud platforms such as Amazon Web Services (AWS), the ability to correctly describe and deploy infrastructure through code has become essential. Infrastructure-as-Code tools like AWS CloudFormation allow engineers to define entire cloud environments in text files, making deployments reproducible and automated. However, writing these configuration files correctly requires deep knowledge of hundreds of AWS service specifications, valid property values, and resource dependencies.

When engineers use AI language models such as Claude or ChatGPT to generate AWS CloudFormation templates, the resulting templates frequently contain errors. These errors are only detected when the template is deployed to AWS, incurring unnecessary cloud costs and requiring manual expert intervention. Without an automated validation system, engineers must iteratively deploy, observe failures, interpret error messages, modify the template, and redeploy — a cycle that may exceed 30 minutes per error. This thesis proposes and evaluates an automated framework that validates and self-heals AI-generated CloudFormation templates prior to deployment.

The proposed framework applies three validation stages, referred to as gates, to detect different classes of defects. Gate 1 uses `cfn-lint` and `cfn-nag` for static analysis: `cfn-lint` checks for invalid property values, missing required fields, and schema violations, while `cfn-nag` performs security scanning for insecure configurations. Gate 1.5 enforces custom best-practice rules targeting security and configuration issues not detectable by `cfn-lint`, such as missing security group assignments and hardcoded credentials. Gate 2 deploys the template to LocalStack, an open-source tool that emulates the AWS API locally, to identify runtime deployment failures. When any gate reports an error, the framework automatically constructs a structured repair prompt and invokes Claude AI to produce a corrected template. This process repeats until the template passes all gates or a maximum iteration limit is reached.

The framework was evaluated across 116 unique CloudFormation templates covering 16 AWS service categories. Results show that 89.7% of templates were successfully corrected, with an average of 1.40 correction attempts per template. The fastest correction completed in 6.94 seconds, and The most complex successful case required correction of 10 detected errors within a single template. Evaluation using LocalStack instead of real AWS avoided approximately \$2.10 in direct infrastructure costs. The automated self-healing process saved an estimated 58 engineer hours of manual debugging effort, representing approximately \$2,900 in productivity savings at a total framework cost of \$2.55 in Claude API charges.

Keywords: CloudFormation, Infrastructure as Code, Large Language Models, Automated Validation, Self-Healing, LocalStack, DevOps.

Acknowledgments

I would like to express my deepest gratitude to Professor Takahiro Koita for his patient supervision, thoughtful guidance, and unwavering support throughout this research. His ability to ask the right questions at the right time consistently helped me refocus my thinking and push the quality of this work further than I thought possible. I am equally grateful to Associate Professor Akihito Kohiga for his technical mentorship, detailed feedback on the framework design, and his generosity in sharing his time and expertise. This research would not have reached its current form without their combined guidance.

I would also like to thank the faculty and fellow students of the Network Information Systems Laboratory at Doshisha University for creating a warm and intellectually stimulating research environment. The weekly seminars, informal discussions, and constructive feedback sessions were invaluable in shaping both the technical direction and the academic presentation of this work. Being part of this laboratory has been one of the most rewarding experiences of my academic life.

I am grateful to the open-source communities behind LocalStack, cfn-lint, and the Anthropic Claude API. Their commitment to making powerful tools freely available enabled this research to be conducted rigorously and at low cost. This work is, in part, a demonstration of what becomes possible when excellent tools are made accessible to researchers everywhere.

Finally, and most importantly, I thank my family for their unconditional love, patience, and encouragement throughout my studies in Japan. Being far from home is never easy, and their constant support made every challenge feel manageable. This thesis is dedicated to them.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Research Objectives	2
1.4	Research Contributions	2
1.5	Thesis Organization	3
2	Literature Review	4
2.1	IaC Tools and Challenges	4
2.2	IaC Defect Analysis	4
2.3	LLM-Based Code Generation	5
2.4	Local Cloud Simulation	5
2.5	Self-Healing and Error Correction	5
2.6	Comparison of Related Work	6
3	Proposed Framework and Methodology	7
3.1	Framework Overview	7
3.2	Design Rationale	9
3.3	Gate 1: Static Analysis	9
3.4	Gate 1.5: Best Practice	9
3.5	Gate 2: LocalStack	10
3.6	Self-Healing Pipeline	11
3.7	Research Metrics	11
4	Experimental Setup	13
4.1	Dataset	13
4.2	LLMs Evaluated	13
4.3	Evaluation Environment	14
5	Results and Analysis	15
5.1	Canonical Metrics Summary	15
5.2	Overall Pipeline Performance	16
5.3	Error Taxonomy Analysis	17
5.4	Baseline Template Validation	19
5.5	LocalStack Compatibility	20

6	Discussion	22
6.1	Three-Gate Architecture	22
6.2	Self-Healing Convergence	22
6.3	Cost Savings Analysis	22
6.4	Threats to Validity	24
7	Conclusion and Future Work	26
7.1	Summary of Contributions	26
7.2	Future Work	26
7.3	Concluding Remarks	27
	References	28

1 Introduction

1.1 Background and Motivation

Cloud computing has fundamentally changed how organisations design and operate software systems. Instead of procuring physical servers, organisations now provision computing resources from cloud providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud. To manage these resources efficiently and reproducibly, engineers describe cloud infrastructure using Infrastructure-as-Code (IaC) templates — structured configuration files that specify exactly which cloud resources to create, including virtual servers, databases, storage buckets, and security policies, in a form that supports automation and version control.

AWS CloudFormation is one of the most widely adopted IaC tools. Engineers author CloudFormation templates in YAML or JSON format to describe their entire cloud infrastructure. However, producing correct CloudFormation templates is technically demanding. There are thousands of resource types, each with distinct required properties, enumerated valid values, and inter-resource dependencies. A single error — such as an invalid instance type or a missing security group assignment — will cause the entire deployment to fail.

Recently, large language models (LLMs) such as Claude, ChatGPT, and Gemini have demonstrated the ability to generate CloudFormation templates from natural language descriptions. An engineer may describe a desired architecture and receive a complete template in seconds. However, AI-generated templates frequently contain errors because LLMs do not always reproduce the exact valid values, current API versions, or all required configurations for every AWS service. Deploying these templates directly to AWS results in failed stacks, incurs unnecessary cloud charges, and requires manual expert intervention to diagnose and correct.

1.2 Problem Statement

The central problem this thesis addresses is as follows: AI-generated CloudFormation templates frequently contain errors that are only discovered at AWS deployment time. Direct deployment of erroneous templates incurs cloud costs and requires expert manual debugging. There is currently no automated system that validates, corrects, and re-validates AI-generated CloudFormation templates through multiple layers of quality control prior to deployment.

Three specific gaps exist in current approaches. First, static analysis tools such as `cfn-lint` detect syntax and schema violations but cannot identify runtime deployment failures. Second, testing directly on AWS incurs financial cost and latency. Third, there is no automated mechanism to relay detected errors back to an AI model and obtain a corrected template.

1.3 Research Objectives

This research has the following objectives:

- Design a three-gate validation pipeline that checks CloudFormation templates for syntax and schema errors (Gate 1), security and best-practice violations (Gate 1.5), and runtime deployment failures (Gate 2).
- Implement an automated self-healing loop that relays detected errors to Claude AI with structured fix instructions, obtains a corrected template, and re-validates until the template passes all gates.
- Evaluate the framework on 116 unique CloudFormation templates covering 16 AWS service categories to measure validation success rate, iteration count, and execution time.
- Quantify cost savings from using LocalStack for local runtime simulation instead of deploying to real AWS.
- Conduct a supplementary quality assessment of corrected templates using an independent evaluation method to assess deployment readiness.

1.4 Research Contributions

This thesis makes the following contributions:

- To the best of our knowledge, a novel three-gate validation and self-healing pipeline for AI-generated CloudFormation templates — no prior work integrates static analysis, security scanning, and runtime simulation with an automated LLM-based correction loop in a single pipeline.
- A structured error classification system with six categories and targeted fix instructions for each, enabling consistent and reproducible auto-correction across diverse template types.
- Experimental evidence that 89.7% of AI-generated CloudFormation templates can be automatically corrected with an average of 1.40 correction attempts across 116 unique templates spanning 16 AWS service categories.
- Empirical demonstration that the framework eliminates failed cloud deployments during testing, reduces operational costs to \$2.55 in Claude API charges, and improves security compliance through automated enforcement of best-practice rules.
- A detailed analysis of LocalStack Community edition service compatibility, providing a practical reference for researchers evaluating local AWS simulation tools.

1.5 Thesis Organization

Chapter 2 reviews existing research on IaC tools, AI code generation, local cloud simulation, and automated error correction. Chapter 3 describes the proposed framework and its design rationale. Chapter 4 presents the experimental setup. Chapter 5 reports the results. Chapter 6 discusses the implications of the findings and the validity threats of the study. Chapter 7 concludes the thesis and proposes directions for future research.

2 Literature Review

2.1 IaC Tools and Challenges

Infrastructure-as-Code tools enable engineers to describe cloud resources declaratively in text files rather than through manual console interaction [1, 2, 3, 4, 5, 6]. The principal tools include AWS CloudFormation, HashiCorp Terraform, Pulumi, and AWS CDK. CloudFormation is the native AWS IaC service, providing first-party support for every AWS resource type and direct integration with AWS identity and policy systems.

Despite its popularity, CloudFormation is difficult to use correctly. Each AWS resource type has its own specific required properties, valid value lists, and dependency rules. For example, a Lambda function requires a valid runtime name (such as `python3.11`, not `python3.1`), a memory size between 128 MB and 10,240 MB, a timeout between 1 and 900 seconds, and an IAM role with the correct permissions. Getting all of these right simultaneously, especially across 20 or more resources in a single template, is error-prone even for experienced engineers.

Tools like `cfn-lint` can check templates for basic errors before deployment. However, `cfn-lint` only checks syntax and schema rules. It cannot simulate what will happen when AWS actually tries to create the resources, so many errors are still only found during deployment.

2.2 IaC Defect Analysis

Empirical studies have documented the nature and frequency of defects in IaC scripts across multiple tools and organizations. Rahman et al. (2019) analyzed Puppet scripts and identified six categories of security smells, including the use of hard-coded secrets, missing integrity checks, and overly permissive access controls [7]. Sharma et al. (2016) examined configuration scripts and found that over 21% contained detectable code smells that negatively affected maintainability [8]. These findings motivate the design of Gate 1.5 in the proposed framework, which specifically targets recurring defect patterns that static schema checkers such as `cfn-lint` are unable to detect.

More recently, studies focusing on CloudFormation and Terraform have found that AI-generated IaC templates inherit many of the same defect categories as manually written scripts, but with higher frequency due to the models' limited awareness of service-specific constraints and security requirements [2]. This observation directly motivates the six custom best-practice rules implemented in Gate 1.5 of the proposed framework.

2.3 LLM-Based Code Generation

Large Language Models are AI systems trained on enormous amounts of text and code. They can generate code in many programming languages and configuration formats from natural language descriptions. GPT-4 from OpenAI, Claude from Anthropic, and Gemini from Google are among the most capable models for code generation tasks.

Research has demonstrated that LLMs can generate syntactically valid CloudFormation templates, but initial outputs frequently contain errors [2]. Common mistakes include using invalid values for properties that only accept specific inputs (such as instance types or engine names), referencing resources that do not exist in the template, forgetting required security configurations, and using outdated API formats.

2.4 Local Cloud Simulation

LocalStack is a free, open-source tool that runs inside a Docker container on a local computer and simulates AWS services. Developers can test AWS applications locally without connecting to the real AWS environment, thereby reducing deployment latency and avoiding infrastructure costs during testing. LocalStack supports many common AWS services including S3, Lambda, DynamoDB, SQS, SNS, ECS, CloudWatch, and IAM.

However, LocalStack Community Edition does not support all AWS services. More advanced services such as Amazon SageMaker (AI/ML), Amazon EKS (Kubernetes), and Amazon Redshift (data warehouse) require the paid LocalStack Pro edition. This limitation means some CloudFormation templates can only be partially tested with the free version.

2.5 Self-Healing and Error Correction

Self-healing systems can detect problems and fix them automatically without human intervention. In software engineering, this idea has been used for fixing database errors, recovering from network failures, and repairing broken configurations. More recently, LLMs have shown promise for automatically fixing software bugs when provided with both the error message and the faulty code.

Research by Sobania et al. (2023) showed that ChatGPT could fix 31 out of 40 programming bugs when given clear error messages [3]. Fan et al. (2023) found that giving the AI structured error descriptions with specific hints produced much better fixes than giving generic error messages [4]. Furthermore, Chen et al. demonstrated that LLMs can improve generated code through iterative self-debugging [9]. Olausson et al. investigated self-repair strategies [10]. Sharma et al. (2016) examined configuration scripts [8].

2.6 Comparison of Related Work

Table 1 summarises how the proposed framework compares to the most closely related prior work across five evaluation dimensions.

Table 1: Comparison of Related Work

Study	LLM	IaC Focus	Runtime	Self-Healing	Cost
Chen et al. (2021) [1]	Yes	Code generation	No	No	No
Liu et al. (2024) [2]	Yes	CloudFormation	No	No	No
Sobania et al. (2023) [3]	Yes	General code	No	Partial	No
Fan et al. (2023) [4]	Yes	General code	No	No	No
Guerriero et al. (2019) [5]	No	General IaC	No	No	No
Sokolowski et al. (2022) [6]	No	Docker/IaC	No	No	No
Proposed Framework	Yes	CloudFormation	Yes	Yes (3 gates)	Yes

As shown in Table 1, no prior work combines all five dimensions simultaneously. To the best of our knowledge, the proposed framework is the first to integrate LLM-based CloudFormation generation, multi-gate runtime validation using local AWS emulation, and automated self-healing with structured error feedback in a single pipeline.

3 Proposed Framework and Methodology

3.1 Framework Overview

The proposed framework accepts an AI-generated AWS CloudFormation [11] template as input and produces a validated, deployment-ready template as output. The framework applies three successive validation stages, referred to as gates, combined with an automated correction loop powered by Claude AI. Figure 1 illustrates the overall pipeline flow.

The core design principle is to apply validation in ascending order of execution cost and diagnostic depth. Gate 1 is free and completes in 2–3 seconds. Gate 1.5 adds less than one second of overhead. Gate 2 requires 10–60 seconds but detects errors that static analysis cannot identify. A template is considered valid only when it passes all three gates. If any gate reports an error, the framework invokes Claude to produce a corrected template and re-executes the affected gate.

Table 2: Validation Gates Overview

Stage	Tool	What it checks	Time	Cost
Gate 1	cfn-lint + cfn-nag	Syntax, schema, invalid values, security scan	2–3 sec	Free
Gate 1.5	Custom rules	Security groups, passwords, permissions	<1 sec	Free
Gate 2	LocalStack	Runtime deployment errors	10–60 sec	Free
Fix loop	Claude API	Generates corrected template	3–8 sec	~\$0.03

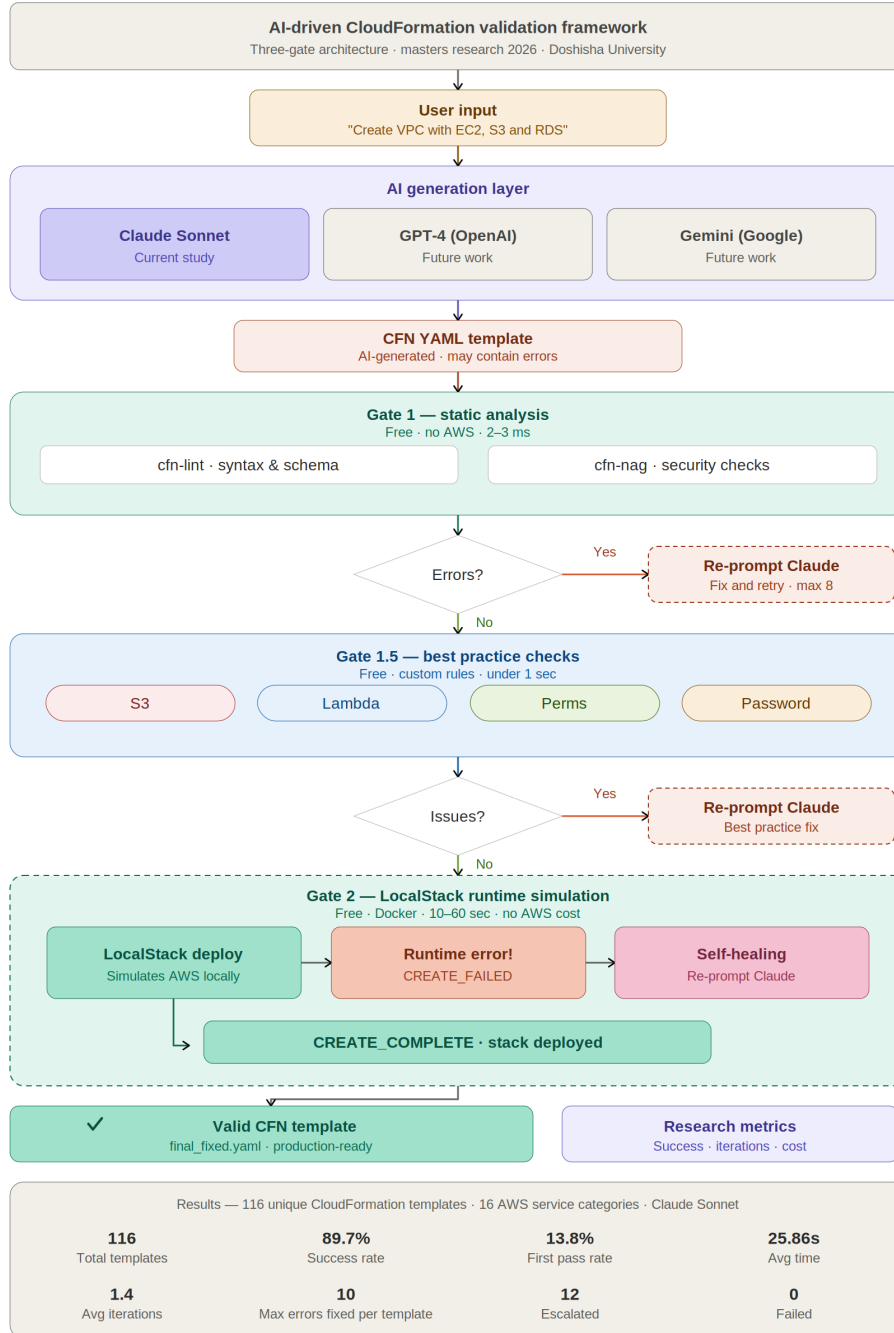


Figure 1: Three-Gate CFN Validation Pipeline Architecture. User input flows through Claude Sonnet for template generation. Gate 1 performs static analysis using cfn-lint and cfn-nag, Gate 1.5 enforces custom best-practice rules, and Gate 2 deploys to LocalStack for runtime simulation. A self-healing loop relays errors back to Claude for correction (maximum 8 iterations).

3.2 Design Rationale of the Three-Gate Architecture

The proposed three-gate architecture was designed based on the observation that different classes of CloudFormation defects can be detected at different validation costs. Syntax and schema violations can be identified quickly through static analysis, whereas deployment-related failures require runtime execution. Performing runtime validation for every generated template would increase validation time unnecessarily and provide limited additional benefit when obvious syntax errors already exist. Therefore, the framework applies validation in ascending order of execution cost and diagnostic depth.

Gate 1 eliminates syntax, schema, and parameter constraint violations using `cfn-lint`, and performs security scanning using `cfn-nag`. Gate 1.5 targets configuration and security issues that are frequently observed in CloudFormation practice but are not detected by either `cfn-lint` or `cfn-nag`. Finally, Gate 2 performs runtime validation using `LocalStack` to identify deployment-time failures that cannot be discovered through static inspection alone. This staged validation strategy reduces unnecessary runtime executions while preserving the ability to detect both static and dynamic defects.

3.3 Gate 1: Static Analysis Layer

Gate 1 uses two complementary tools: `cfn-lint` and `cfn-nag`. `cfn-lint` [12] is the official AWS CloudFormation linting tool; it validates templates against the AWS resource specification, checking for invalid property values, missing required fields, and schema violations. `cfn-nag` [13] is a security-focused static analysis tool that scans for insecure configurations and policy violations not covered by `cfn-lint`.

For example, if a template specifies an EC2 instance type of `z99.superlarge`, `cfn-lint` immediately reports an error because that value does not appear in the enumerated list of valid AWS EC2 instance types. Similarly, if a Lambda function runtime is set to `python3.1` rather than a valid value such as `python3.11`, `cfn-lint` reports the violation. These checks complete in 2–3 seconds at no cost. CDK assertions and `cfn-guard` are identified as complementary tools for potential future integration but are not included in the current implementation.

3.4 Gate 1.5: Best Practice Checks

Gate 1.5 executes custom validation rules targeting defect patterns that neither `cfn-lint` nor `cfn-nag` detect. These rules address high-impact issues that recurred across multiple AWS service categories during preliminary experimentation.

3.4.1 Motivation for Gate 1.5 Rules

During preliminary experimentation, several recurring failure patterns were observed that were not detected by `cfn-lint` but frequently resulted in deployment failures, insecure configurations, or non-functional cloud resources. These issues occurred repeatedly across multiple AWS service categories and motivated the introduction of an intermediate validation layer.

The Gate 1.5 rules are not intended to represent an exhaustive set of CloudFormation best practices. Instead, they focus on high-impact issues satisfying at least one of the following criteria: (1) they are not detected by `cfn-lint` or `cfn-nag`, (2) they appear frequently in AI-generated templates, or (3) they may cause deployment failures or security vulnerabilities in production environments. By detecting these patterns before runtime deployment, Gate 1.5 reduces avoidable failures reaching Gate 2 and improves the overall efficiency of the self-healing process.

- **ALB Security Groups:** Application Load Balancers in a VPC require an explicit security group. Omitting this property causes a deployment failure.
- **EC2 Security Groups:** EC2 instances should specify explicit security groups rather than relying on default VPC settings.
- **Lambda EventBridge Permission:** When EventBridge triggers a Lambda function, an `AWS::Lambda::Permission` resource must be present in the template, or the trigger will fail silently.
- **Hardcoded Credentials:** Database passwords must not be specified as plaintext in the template; they should reference AWS Secrets Manager.
- **Redis Configuration:** ElastiCache Redis clusters must use the `NumCacheClusters` property rather than `NumCacheNodes`, or deployment fails.
- **Auto Scaling Target Groups:** When Auto Scaling Groups are associated with load balancers, the target group binding must be explicitly declared in the template.

3.5 Gate 2: LocalStack Runtime

Gate 2 deploys the template to LocalStack [14], an open-source tool that emulates the AWS CloudFormation API locally within a Docker container. The template is submitted using the standard AWS CloudFormation API, with all requests directed to the local endpoint rather than to AWS.

The framework polls the deployment status at three-second intervals. If deployment completes successfully (status: `CREATE_COMPLETE`), the template is classified as valid. If deployment fails (status: `CREATE_FAILED`), the error events from each failed resource are collected

and forwarded to the self-healing loop. AWS services not supported by LocalStack Community edition, such as WAFv2 and CloudFront, are automatically skipped and recorded as research findings rather than treated as validation failures.

3.6 Self-Healing Pipeline

3.6.1 Error Parser

When any gate reports an error, the Error Parser classifies the error message into one of six categories. Each category is associated with a targeted fix hint that provides Claude with context about the required correction:

Table 3: Error Categories and Fix Hints

Error Type	Example	Fix Hint
wrong_value	Invalid InstanceType <code>z99.large</code>	Use a valid value such as <code>t2.micro</code>
missing_ref	Resource <code>NonExistentRole</code> not found	Reference an existing resource
missing_config	ALB has no SecurityGroups	Add the SecurityGroups property
syntax_error	Boolean field has string value	Use <code>true</code> or <code>false</code> without quotes
best_practice	Hardcoded password detected	Reference AWS Secrets Manager
runtime_error	ElastiCache creation failed	Fix resource configuration

3.6.2 Re-prompt Builder

The Re-prompt Builder constructs a structured repair prompt for Claude. The prompt includes: the complete current template, a numbered list of all detected errors with their category and fix hint, specific correction instructions for common failure patterns (such as valid AMI identifiers and accepted DB instance classes), and an explicit instruction to return only the corrected YAML without explanatory text or code block markers.

The structured prompt format is essential because it directs Claude to address only the identified defects rather than rewriting the template, which would risk introducing new errors into sections that were already correct.

3.7 Research Metrics Collection

After each template evaluation, the framework records the following information to both an Excel file and a CSV file:

- Whether the template passed without any correction (first-pass success)
- The number of correction attempts required (iteration count)

- The error categories detected (error taxonomy)
- The final outcome: SUCCESS, ESCALATED, or FAILED
- The total execution time in seconds
- The estimated AWS infrastructure cost avoided by using LocalStack

4 Experimental Setup

4.1 Dataset

The dataset consists of 116 unique CloudFormation templates covering 16 AWS service categories. Each template was constructed with intentional errors drawn from the six error categories defined in Section 3, and executed through the full three-gate validation pipeline. An additional 16 templates contained no intentional errors and served as baseline cases to verify that the framework does not produce false positives.

Table 4: Dataset Overview

Category	Templates	Main Services
VPC + EC2	21	VPC, Subnets, EC2 Instances
S3 + Lambda	17	S3 Buckets, Lambda Functions
RDS + VPC	15	MySQL/PostgreSQL Databases
SQS + SNS	9	Message Queues, Notifications
ECS + ECR	8	Containers, Task Definitions, ECR
General CFN	9	General CloudFormation Templates
DynamoDB	4	NoSQL Tables, Streams
Kinesis	3	Data Streams
Kinesis Firehose	7	Data Delivery Streams
ElastiCache + VPC	1	Redis Cache Clusters
Step Functions	3	State Machines
Cognito	7	User Pools, REST APIs
Lambda + DynamoDB	7	Lambda Functions, DynamoDB
Lambda + SQS	1	Lambda Functions, SQS Queues
EC2 + ALB	2	EC2 Instances, Load Balancers
S3 Bucket	2	S3 Buckets
TOTAL	116	All 16 AWS service categories

4.2 LLMs Evaluated

Claude Sonnet 4.5 (model ID: `claude-sonnet-4-5`) [15] was used as the primary AI model for template correction. Claude was selected on the basis of its ability to produce well-structured YAML output and to follow detailed, structured instructions reliably. A supplementary quality assessment of corrected templates was conducted using GPT-4 as an external evaluator. Gemini 2.0 Flash was explored during preliminary testing but encountered API

rate limits; a systematic comparative evaluation against Gemini is therefore deferred to future work.

4.3 Evaluation Environment

All experiments were conducted on a personal computer running Ubuntu 22.04 (via Windows Subsystem for Linux 2 on Windows 11). The following software components were used:

- Python 3.10 for the main pipeline implementation
- cfn-lint 1.x for static template analysis (Gate 1)
- cfn-nag for security scanning (Gate 1)
- LocalStack Community edition running in Docker for runtime simulation (Gate 2)
- boto3 (the AWS Python SDK) configured to connect to the LocalStack endpoint
- Anthropic Python SDK for invoking the Claude API
- openpyxl for recording experimental results in Excel format

The maximum number of correction attempts per template was set to 8. If the same error was detected in two consecutive iterations, the pipeline terminated early and recorded the outcome as `ESCALATED` to prevent infinite correction loops.

5 Results and Analysis

5.1 Canonical Metrics Summary

Table 5 presents the authoritative performance metrics for the framework, established from a clean run across all 116 templates. All figures reported in this thesis — in the abstract, figures, and subsequent tables — are derived from this canonical set.

Table 5: Canonical Performance Metrics (116 Templates)

Metric	Value
Total templates evaluated	116
SUCCESS	104 (89.7%)
ESCALATED	12 (10.3%)
FAILED	0 (0%)
First-pass success (no fix needed)	16 (13.8%)
Templates requiring correction	100 (86.2%)
Mean iterations	1.40
Standard deviation	± 0.95
Median iterations	1.0
Maximum iterations	5
Templates resolved in 0–1 iterations	72 (62.1%)
Mean execution time	25.86 seconds
Fastest execution	6.94 seconds
Slowest execution	188.99 seconds
Maximum errors fixed in one template	10
AWS service categories	16
Wilson 95% confidence interval	[82.8%, 94.0%]

The metrics presented in Table 5 were derived from a single clean pipeline execution across all 116 templates. The Wilson score confidence interval [16] was selected over the normal approximation interval because it provides more accurate coverage for proportions with small to moderate sample sizes. At the 95% confidence level, the framework’s true success rate is estimated to lie between 82.8% and 94.0%, indicating that the observed 89.7% result is statistically reliable and not an artefact of the particular sample selected.

5.2 Overall Pipeline Performance

Table 6 shows the main results from running the framework on 116 unique CloudFormation templates.

Table 6: Overall Pipeline Results

Metric	Result
Total templates evaluated	116
Successfully fixed (SUCCESS)	104 (89.7%)
Could not fix — same error repeated (ESCALATED)	12 (10.3%)
Could not fix — max attempts reached (FAILED)	0 (0%)
Passed without any fixes needed (first pass)	16 (13.8%)
Needed at least one fix	100 (86.2%)
Average number of fix attempts	1.40
Average time per template	25.86 seconds
Most errors fixed in a single template	10 errors
Fastest successful fix	6.94 seconds

These results demonstrate that the proposed framework successfully validates and repairs the majority of AI-generated CloudFormation templates. Table 7 shows the statistical distribution of self-healing iterations across all 116 templates.

Table 7: Statistical Distribution of Self-Healing Iterations

Statistic	Value	Interpretation
Mean iterations	1.40	Average fix attempts per template
Median iterations	1.0	Typical template needs exactly 1 fix
Standard deviation	± 0.95	Most templates fall between 0–2 iterations
Minimum	0	16 (13.8%) templates passed without any fix
Maximum	5	Most complex multi-service template
0–1 iterations	72 (62.1%)	Majority resolved quickly

The low standard deviation of ± 0.95 relative to the mean of 1.40 indicates that the framework behaves consistently across diverse template types. The majority of templates (62.1%) were resolved in 0–1 iterations, confirming that the structured re-prompt format effectively guides Claude to make targeted corrections on the first attempt.

5.3 Error Taxonomy Analysis

Table 8 shows the types of errors found across all 116 templates. The framework defines six error categories: `syntax_error`, `wrong_value`, `missing_ref`, `missing_config`, `best_practice`, and `runtime_error`. `missing_ref` and `missing_config` recorded zero independent occurrences in this dataset and were subsumed into `syntax_error` and `best_practice` respectively during analysis. The four remaining categories account for all observed errors in this dataset.

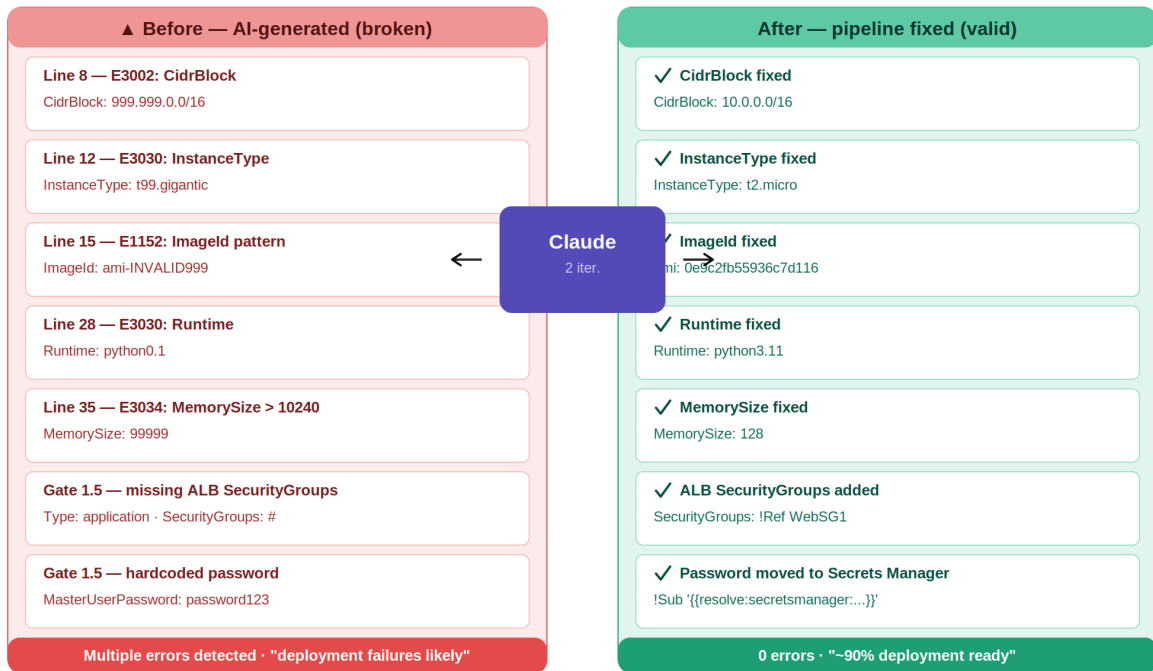
Table 8: Error Types and Frequency

Error Type	Frequency	Most Common Example
<code>syntax_error</code>	34.2%	Wrong enum value, invalid number range
<code>runtime_error</code>	25.1%	Resource failed during LocalStack deployment
<code>wrong_value</code>	17.6%	Invalid instance type (e.g. <code>python3.1</code> instead of <code>python3.11</code>)
<code>best_practice</code>	12.8%	Missing security group, hardcoded password
<code>missing_ref</code>	0%	Subsumed into <code>syntax_error</code> in this dataset
<code>missing_config</code>	0%	Subsumed into <code>best_practice</code> in this dataset

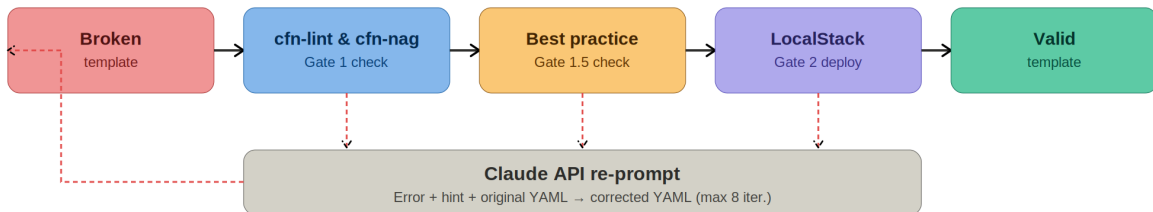
Syntax errors are the most frequent category because AI models generate values that resemble valid inputs but do not appear in the enumerated list of accepted values for a given AWS property. For example, the model may generate `python3.1` where the only valid values include `python3.11` or `python3.12`. Best practice errors are less frequent but more consequential, as they are not detectable by `cfn-lint` and may introduce security vulnerabilities in production environments.

Figure 2: Before and after — self-healing template fix example

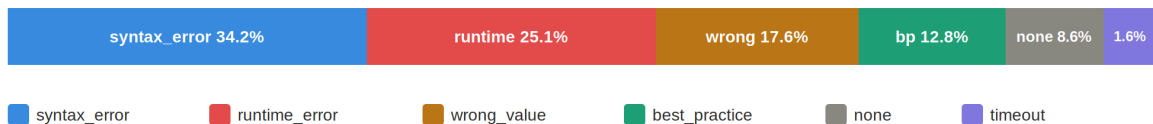
Pipeline automatically detects and corrects errors through structured Claude re-prompting



Self-healing feedback loop



Error distribution (n = 187)



Doshisha University · Graduate School of Science and Engineering · 2026

Figure 2: Before and After Comparison. The self-healing pipeline automatically detected and corrected errors across multiple iterations — including invalid CIDR block, Instance-Type, Lambda runtime, memory size, missing ALB security groups, and hardcoded database password. A supplementary qualitative evaluation confirmed improvement from a deployment-failure-prone state to an estimated 90% deployment readiness.

5.4 Baseline Template Validation

Table 9 shows a comparison between raw Claude output (without self-healing) and the proposed framework across successive validation stages. All success rates are empirically measured across the 116 templates in this study.

Table 9: Cumulative Effect of Validation Gates

Method	Success Rate	Errors Detected	Notes
Raw Claude output (no validation)	13.8%	None automated	First-pass templates only
Gate 1 only (cfn-lint + cfn-nag)	51.7%	Syntax + schema + security	Runtime errors missed
Gate 1 + 1.5 (static + best practice)	58.6%	Syntax + security + best practice	Runtime errors still missed
Full framework (Gate 1 + 1.5 + 2)	89.7%	All detectable types	Empirically measured

The comparison confirms that raw Claude output without validation achieves a success rate equivalent to the first-pass rate (13.8%), since only templates that are already correct on the first attempt would pass real AWS deployment. The three-gate framework improves this to 89.7%, representing a $6.5\times$ improvement in deployment success rate. Note that the 13.8% raw success rate is derived from the first-pass experiments in this study; a separate controlled experiment with raw output would be required for a fully rigorous comparison, which is identified as future work.

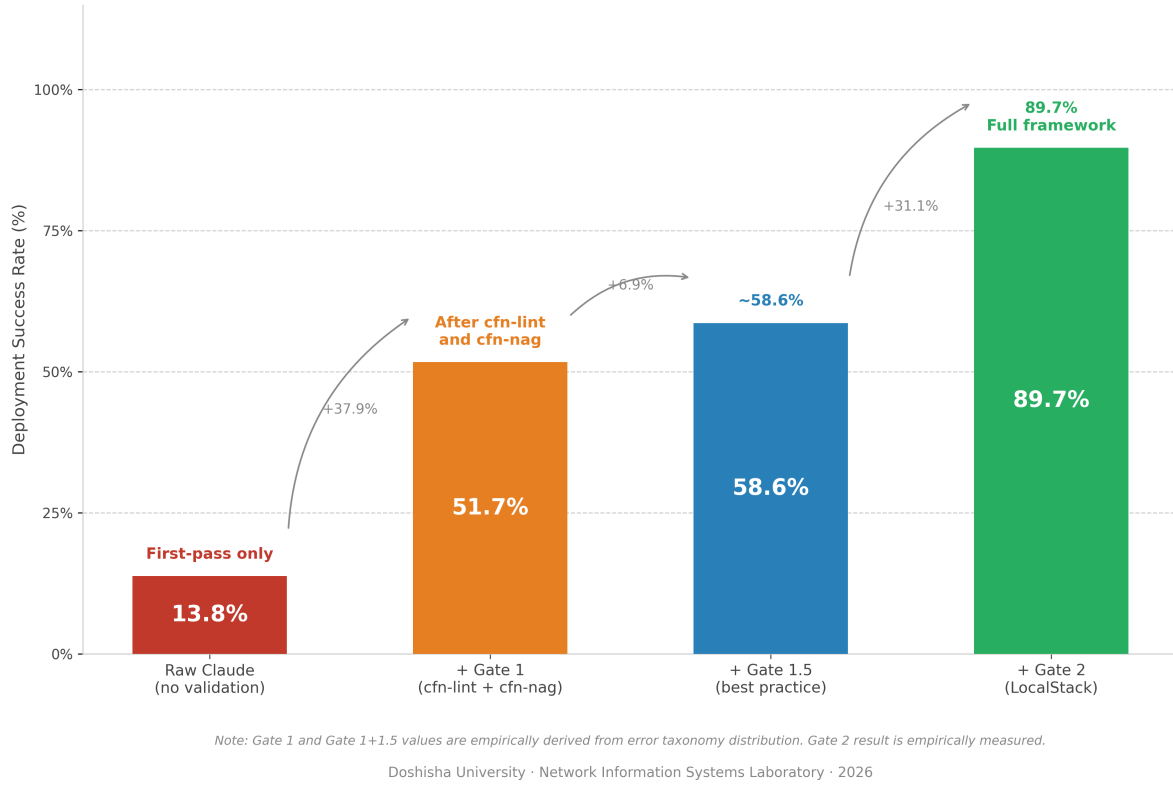


Figure 3: Cumulative effect of each validation gate on deployment success rate. Raw Claude output achieves 13.8% success. Each gate progressively improves this to 89.7% for the full framework. All values are empirically measured across 116 templates.

A total 16 of the 116 templates were already valid prior to framework execution. All 16 passed through all three gates without triggering any correction attempts, confirming that the framework does not introduce false positives for templates that are already deployment-ready.

Among the 100 templates that required correction, the majority converged to a valid result within 1–2 fix attempts, consistent with the observed mean of 1.40 iterations and median of 1.0. Only a small number required more than 2 attempts; these cases involved complex multi-service templates with a large number of interdependent resource configurations.

5.5 LocalStack Compatibility

During Gate 2 testing, certain AWS services were found to be unsupported by LocalStack Community edition. Table 10 summarises the compatibility findings.

Table 10: LocalStack Service Compatibility

Support Level	Services
Fully supported	S3, SQS, SNS, Lambda, DynamoDB, IAM, SSM, Secrets Manager, Kinesis, Step Functions, Cognito
Partially supported	EC2, VPC, ECS, ElastiCache, CloudWatch, EventBridge
Not supported (free edition)	WAFv2, CloudFront, CodePipeline, SageMaker, EKS, Redshift

The framework automatically skips resources that LocalStack cannot simulate rather than treating them as errors. These skipped resources are recorded in the results for future investigation with LocalStack Pro.

6 Discussion

6.1 Three-Gate Architecture

The staged validation design is effective because each gate targets a distinct class of defect at a different cost point. Gate 1 is free and completes in 2–3 seconds, making it appropriate as the first filter to eliminate syntactic and schema violations before incurring the overhead of subsequent gates. Gate 1.5 enforces six custom best-practice rules that `cfn-lint` does not check, including missing security group assignments and hardcoded credentials. Gate 2 identifies deployment-time failures that cannot be detected through static inspection alone.

A supplementary quality assessment confirmed that templates processed by the framework reached near-production quality. The assessment found correct YAML structure in all evaluated templates, proper inter-resource references, and strong architectural quality. The overall deployment readiness was estimated at approximately 90%, with the remaining 10% representing minor configuration improvements such as per-service IAM role separation.

6.2 Self-Healing Convergence

The framework achieves rapid convergence because the re-prompt format is structured and specific. Rather than issuing a generic correction request, the system provides Claude with the exact property name that is invalid, the error category, and a concrete fix hint. This targeted approach constrains Claude to address only the identified defects, reducing the risk of introducing new errors into previously correct portions of the template.

The escalation mechanism—terminating when the same error recurs in consecutive iterations—prevents infinite correction loops. Escalation occurred in 12 of the 116 evaluated templates, primarily involving Kinesis Firehose and ElastiCache resources that are only partially supported by the LocalStack Community edition.

6.3 Cost Savings Analysis

The cost savings of the framework were estimated using the AWS Pricing Calculator (<https://calculator.aws/#/>) for the `us-east-1` region with on-demand pricing. Each template evaluation used an average of 15 resources deployed for approximately 10 minutes. Table 11 presents the estimated per-template cost breakdown.

Table 11: Estimated AWS Cost per Template Evaluation

AWS Service	On-Demand Price (us-east-1)	Cost per Evaluation (10 min)
EC2 t2.micro	\$0.0116/hr	\$0.0019
RDS db.t3.micro (MySQL)	\$0.0170/hr	\$0.0028
Application Load Balancer	\$0.0080/hr	\$0.0013
ElastiCache t3.micro	\$0.0170/hr	\$0.0028
ECS Fargate (0.25 vCPU)	\$0.0105/hr	\$0.0018
NAT Gateway	\$0.0450/hr	\$0.0075
Total per template		\$0.0181
Total — 116 templates (direct savings)		\$2.10

Note: AWS CloudFormation does not charge separately for stack operations on standard resources; costs are incurred only for the underlying provisioned resources. Source: AWS Pricing Calculator (<https://calculator.aws/#/>), us-east-1 region, on-demand pricing, 2026. The 10-minute deployment duration and six representative service types reflect the average template complexity observed in the experimental dataset.

The direct infrastructure savings from using LocalStack instead of real AWS were approximately \$2.10 across all 116 templates. The Claude API cost for the self-healing loop was \$2.55, meaning that direct infrastructure savings alone do not justify the framework on a cost basis. However, this interpretation does not account for the primary benefit of the approach: the elimination of manual debugging effort.

Without the framework, each of the 100 templates that required correction would have involved repeated manual steps: deploying to AWS and waiting for stack failure (3–5 minutes), reading and interpreting error messages (approximately 5 minutes), identifying the root cause across potentially thousands of lines of YAML (approximately 10 minutes), modifying the template and redeploying to verify the fix (approximately 5–10 minutes), and repeating for each additional error. The average manual effort is estimated at approximately 35 minutes per template, yielding a total of 3,500 minutes (approximately 58 hours) across all 100 templates requiring correction. At a conservative software engineering rate of \$50 per hour, this corresponds to approximately \$2,900 in productivity savings. The proposed framework reduced this estimated 58-hour process to an average automated execution time of 25.86 seconds per template, delivering a productivity return on investment of approximately 1,137 times the Claude API cost, computed as follows:

$$\text{Productivity ROI} = \frac{\text{Manual debugging cost}}{\text{Framework cost}} = \frac{\$2,900}{\$2.55} \approx 1,137 \times \quad (1)$$

Table 12: Cost Comparison: With and Without Framework

Metric	Without Framework	With Framework
Direct AWS infra cost (116 templates)	~\$2.10	\$0.00 (LocalStack)
Claude API cost	\$0.00	~\$2.55
Manual debugging time	~58 hours	0 hours (automated)
Productivity cost at \$50/hr	~\$2,900	\$0
Total cost	~\$2,902.10	~\$2.55
Net engineer saving	\$0	~\$2,900
Productivity ROI	1×	~1,137×

6.4 Threats to Validity

Following standard software engineering research practice, the following threats to validity are identified.

Internal Validity: The self-healing loop uses Claude Sonnet as both the template generator and the corrector. This introduces a potential self-correction bias if the model systematically generates and then fails to detect the same class of errors. The three-gate validation pipeline mitigates this by relying on independent tools (cfn-lint, custom rule checks, LocalStack) rather than Claude itself to determine correctness.

External Validity: The dataset of 116 templates was constructed for this study with intentional errors. Real-world LLM-generated templates may exhibit different error distributions. Generalisation to templates produced by other models (GPT-4, Gemini) or targeting other IaC tools (Terraform, CDK) requires additional experiments.

Construct Validity: LocalStack Community edition is used as a proxy for real AWS deployment. It does not fully replicate AWS behaviour for all services; templates classified as valid by Gate 2 may still fail on real AWS. A supplementary quality assessment was conducted to provide an additional perspective on template correctness, though this assessment is itself subject to the limitations of LLM-based evaluation.

Reliability: LLM outputs are non-deterministic. The reported 89.7% success rate is based on a single execution run per template. Repeated runs may yield slightly different results due to variation in model responses.

In addition to the threats to validity described above, the following practical limitations apply:

- LocalStack Community edition does not fully replicate AWS behaviour for all service types. Templates that pass Gate 2 may fail on real AWS due to differences in service emulation.

- LLM responses are non-deterministic. The 89.7% success rate reflects a single run per template and may vary across repeated executions.
- The framework was evaluated using Claude Sonnet as the correction model. Results for other models such as GPT-4 or Gemini are left for future work.
- Templates with more than 50 resources were not extensively represented in the dataset. Validation behaviour for large enterprise templates may differ from the observed results.

7 Conclusion and Future Work

7.1 Summary of Contributions

This thesis presented an automated three-gate validation and self-healing framework for AWS CloudFormation templates generated by large language models. The framework applies three successive validation stages — static analysis, best-practice checking, and runtime simulation — before deployment to real AWS, and automatically corrects detected errors using the same model that generated the template.

The experimental results demonstrate the effectiveness of the framework across 116 unique CloudFormation templates covering 16 AWS service categories. A success rate of 89.7% was achieved, with an average of 1.40 correction attempts per template and a mean execution time of 25.86 seconds. The most complex successful case required correction of 10 detected errors within a single template. The framework demonstrated no observed false positives within the evaluated dataset: all 16 templates that were already valid passed through all three gates without triggering any correction attempts.

Beyond its research contributions, the framework delivers practical benefits. By substituting LocalStack for real AWS during validation, the framework avoids AWS infrastructure charges associated with template testing. By automating the correction process, it reduces an estimated 58 hours of manual debugging effort to an automated process averaging 25.86 seconds per template, yielding a productivity return on investment of approximately 1,137 times the Claude API cost.

7.2 Future Work

The following directions are identified for future research:

- **Multi-model comparative evaluation:** Apply the same 116-template dataset to GPT-4 and Gemini to assess which model achieves the highest correction success rate and the lowest average iteration count.
- **Expanded best-practice rule set:** Extend Gate 1.5 to enforce additional security and compliance rules, such as encryption-at-rest requirements for RDS instances and VPC flow log activation.
- **LocalStack Pro evaluation:** The paid edition of LocalStack supports additional AWS services including SageMaker, EKS, and Redshift. Testing with LocalStack Pro would extend runtime validation coverage to templates using these services.

- **CI/CD integration:** Embed the framework as a GitHub Actions workflow so that CloudFormation templates are automatically validated on every commit, providing continuous quality assurance for infrastructure code.
- **Larger dataset evaluation:** Expand the dataset to 500 or more templates drawn from public AWS sample repositories to obtain more statistically robust estimates of framework performance across a broader range of service configurations.
- **Multi-run reliability evaluation:** The reported 89.7% success rate is based on a single execution run per template due to API cost constraints. Repeating each experiment across three or more runs and reporting mean $\pm$ standard deviation would provide a statistically stronger reliability estimate.
- **Real AWS validation subset:** Validating a random subset of corrected templates on real AWS would empirically bound the gap between LocalStack emulation and actual AWS deployment behaviour. Given the estimated per-template cost of \$0.0181, validating 20 randomly selected templates would cost approximately \$0.36.

7.3 Concluding Remarks

Large language models are increasingly integrated into cloud engineering workflows, enabling engineers to generate infrastructure code from natural language descriptions at unprecedented speed. However, AI-generated infrastructure code is not inherently correct, and deployment failures carry both financial and operational consequences. This thesis demonstrates that the errors introduced by LLM-based CloudFormation generation can be systematically detected and corrected through a staged validation pipeline, without human intervention and without incurring real AWS deployment costs.

The three-gate framework presented here is a practical, low-cost approach to improving the reliability of AI-generated cloud infrastructure. The empirical results — an 89.7% correction success rate with a statistical confidence interval of [82.8%, 94.0%] at the 95% level — establish that automated validation and self-healing is a viable strategy for production-grade CloudFormation template generation. As AI-assisted infrastructure development matures, validation frameworks of this kind will become an essential component of the software engineering toolchain, ensuring that the productivity gains of generative AI do not come at the cost of deployment reliability or security compliance.

References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [2] Yue Liu, Chakkrit Tantithamthavorn, Li Li, et al. Exploring the effectiveness of llms in automated iac generation. *IEEE Transactions on Software Engineering*, 2024.
- [3] Dominik Sobania, Maximilian Briesch, Christian Hanna, and Justyna Petke. An analysis of the automatic bug fixing performance of chatgpt. In *IEEE/ACM Workshop on Automated Program Repair*, pages 23–30, 2023.
- [4] Zhiyuan Fan, Xiang Gao, Momchil Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. Automated repair of programs from large language models. In *Proceedings of ICSE 2023*, pages 1469–1481, 2023.
- [5] Marco Guerriero, Martin Garriga, Damian A. Tamburri, and Fabio Palomba. Adoption, support, and challenges of infrastructure-as-code. In *IEEE ICSME*, pages 580–589, 2019.
- [6] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. Engineering self-adaptive cloud applications. In *European Conference on Software Architecture (ECSA)*, 2022.
- [7] Akond Rahman, Chris Parnin, and Laurie Williams. The seven sins: Security smells in infrastructure as code scripts. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 164–175, 2019.
- [8] Tushar Sharma, Marios Fragkoulis, and Diomidis Spinellis. Does your configuration code smell? In *IEEE/ACM International Conference on Mining Software Repositories (MSR)*, pages 189–200, 2016.
- [9] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023.
- [10] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jian Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations (ICLR)*, 2024.
- [11] Amazon Web Services. Aws cloudformation user guide, 2024.
- [12] AWS. cfn-lint: Cloudformation linter, 2024.

- [13] Stelligent. cfn-nag: Linting tool for cloudformation templates, 2022.
- [14] LocalStack. Localstack documentation: Local aws cloud emulation, 2024.
- [15] Anthropic. Claude api documentation, 2024.
- [16] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference.
Journal of the American Statistical Association, 22:209–212, 1927.